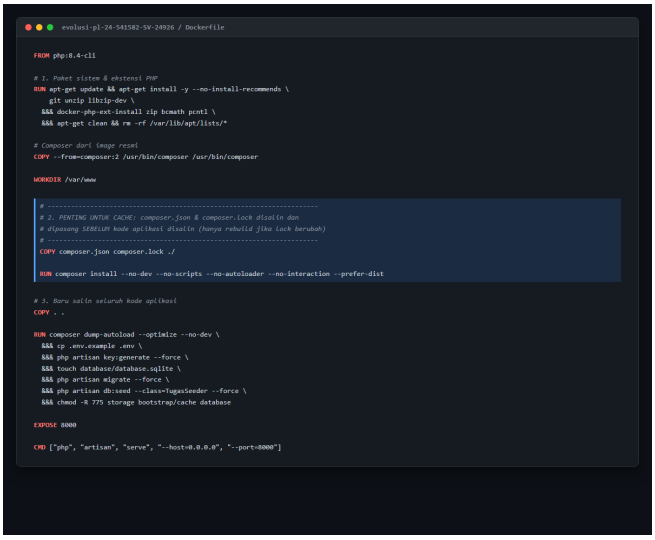
LAPORAN UJIAN TENGAH SEMESTER

KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

Kode Mata Kuliah: SVPL26150 (2 SKS)

Pengembangan Full-Stack REST API Laravel & Vue.js, Container Multi-Stage, CI/CD Pipeline ke GHCR, Docker Compose, dan FrankenPHP



Nama	:	**Hisyam Al Rayyan**
NIM	:	**24/541582/SV/24926**
Program Studi	:	D4 Teknologi Rekayasa Perangkat Lunak
Departemen	:	Teknik Elektro dan Informatika
Sekolah	:	Sekolah Vokasi, Universitas Gadjah Mada
Dosen Pengampu	:	Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
		Galih Malela Damaraji, S.Pd., M.Eng.

Yogyakarta, 2026

Contents

1 Pendahuluan & Identitas Repositori **2**
- 1.1 Deskripsi Proyek Ujian . . . 2
- 1.2 Informasi Repositori & Berkas Pengujian . . . 2

2 Soal 1: Backend Laravel RESTful API (CPMK1) **4**
- 2.1 Ketentuan dan Implementasi Soal 1 . . . 4
- 2.2 Bukti Pengujian Backend . . . 5

3 Soal 2: Frontend Vue.js (CPMK1) **9**
- 3.1 Ketentuan dan Implementasi Soal 2 . . . 9
- 3.2 Bukti Tampilan Frontend Vue.js . . . 10

4 Soal 3: Manajemen Repositori GitHub & CI Pipeline (CPMK2) **12**
- 4.1 Ketentuan dan Implementasi Soal 3 . . . 12
- 4.2 Bukti Branch Protection & CI Runs . . . 13

5 Soal 4: Continuous Deployment ke GHCR (CPMK3) **15**
- 5.1 Ketentuan dan Implementasi Soal 4 . . . 15
- 5.2 Bukti Eksekusi CD & Paket Kontainer . . . 16

6 Soal 5: Dockerfile Multi-Stage & Best Practices (CPMK4 & CPMK5) **19**
- 6.1 Ketentuan dan Implementasi Soal 5 . . . 19
- 6.2 Bukti Arsitektur Dockerfile . . . 20

7 Soal 6: Docker Compose Multi-Container (CPMK6 & CPMK7) **21**
- 7.1 Ketentuan dan Implementasi Soal 6 . . . 21
- 7.2 Bukti Status Layanan & Persistensi Volume . . . 22

8 Soal 7: Nilai Tambahan FrankenPHP (+10 Poin) **23**
- 8.1 Ketentuan dan Implementasi Soal 7 . . . 23
- 8.2 Bukti Log Kontainer FrankenPHP . . . 24

9 Penjelasan Aliran Data: Vue ke Laravel ke Basis Data **25**
- 9.1 Diagram & Tahapan Aliran Data . . . 25

10 Kendala yang Dihadapi dan Solusi **27**

11 Kesimpulan **28**

Chapter 1

Pendahuluan & Identitas Repositori

1.1 Deskripsi Proyek Ujian

Proyek ini dikembangkan untuk memenuhi seluruh Capaian Pembelajaran Mata Kuliah (CPMK) 1 sampai 7 serta nilai bonus (+10) pada mata kuliah Konstruksi dan Evolusi Perangkat Lunak (SVPL26150), Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada.

Tema sistem yang diangkat adalah **Katalog Produk & Inventaris Elektronik (Produk API)**, yang merupakan tema baru dan terpisah dari tugas-tugas sebelumnya (Tugas/Todo List). Sistem ini mengimplementasikan arsitektur berorientasi layanan (*service-oriented architecture*) yang terdiri atas:

- **Backend RESTful API:** Dibangun dengan Laravel 11/13 dan PHP 8.4, mengamankan endpoint privat memakai token Laravel Sanctum, menyediakan CRUD lengkap sumber daya Produk, validasi ketat, serta pengujian otomatis via PHPUnit/Artisan Test.
- **Frontend SPA:** Dibangun dengan Vue 3 (Vite), Axios instance dengan request/response interceptor, state autentikasi token, dan visualisasi katalog responsif.
- **Kontrol Versi & Manajemen Repositori:** Dikelola di GitHub dengan percabangan `main`, `dev`, dan `feature/**`, dilindungi oleh *Branch Protection Rules* yang mewajibkan Pull Request dan kelulusan status check CI.
- **Automasi CI/CD Pipeline:** Menggunakan GitHub Actions untuk menjalankan pengujian backend dan kompilasi frontend, serta melakukan deployment otomatis image Docker ke GitHub Container Registry (`ghcr.io`) dengan tag commit SHA saat branch `main` menerima push.
- **Kontainerisasi & Komposisi:** Dockerfile multi-stage dengan layer caching optimal, berkas `.dockerignore`, serta orkestrasi `docker-compose.yml` (MariaDB, Laravel Backend, Vue Frontend) dengan named volume.
- **Nilai Bonus FrankenPHP:** Backend Laravel dijalankan di atas server modern FrankenPHP (Caddy server terintegrasi PHP worker mode) dengan image berversi jelas.

1.2 Informasi Repositori & Berkas Pengujian

Seluruh kode sumber dan konfigurasi sistem tersedia secara publik pada tautan berikut:

- **Tautan Repositori GitHub:**
  `https://github.com/hisyamalrayyan-lgtm/uts-pl-541582`

- **Berkas Postman Collection:**
  Tersimpan di akar repositori dengan nama `postman_collection.json` serta direktori `postman/UTS_KEPL_Produk_API.postman_collection.json`.
- **GitHub Container Registry (GHCR):**
  - Backend: `ghcr.io/hisyamalrayyan-lgtm/uts-pl-541582-backend`
  - Frontend: `ghcr.io/hisyamalrayyan-lgtm/uts-pl-541582-frontend`

Chapter 2

Soal 1: Backend Laravel RESTful API (CPMK1)

2.1 Ketentuan dan Implementasi Soal 1

Sesuai ketentuan Materi Uji CPMK1 (Nilai 15), backend Laravel menyediakan RESTful API bertema **Katalog Produk** dengan rincian implementasi:

1. **Autentikasi Berbasis Token:** Menggunakan Laravel Sanctum v4. Endpoint otentikasi mencakup `POST /api/register`, `POST /api/login`, `POST /api/logout`, dan `GET /api/me`. Seluruh rute manajemen produk dilindungi oleh middleware `auth:sanctum`.
2. **Resource CRUD Lengkap:** Model `Produk` dan tabel `produks` menyimpan data `nama`, `kategori`, `harga`, `stok`, dan `deskripsi`. Seluruh operasi CRUD diimplementasikan:
   - `GET /api/produks`: Menampilkan seluruh daftar produk.
   - `GET /api/produks/{id}`: Menampilkan rincian satu produk.
   - `POST /api/produks`: Menambahkan produk baru (mengembalikan HTTP 201).
   - `PUT /api/produks/{id}`: Memperbarui data produk (mengembalikan HTTP 200).
   - `DELETE /api/produks/{id}`: Menghapus data produk (mengembalikan HTTP 200).
3. **Format Respon dan Status HTTP yang Sesuai:**
   - `201 Created`: Saat data produk berhasil dibuat.
   - `422 Unprocessable Content`: Saat validasi gagal (misalnya kolom nama kosong atau harga bernilai negatif).
   - `401 Unauthorized`: Saat request ke endpoint terlindungi dikirim tanpa token atau dengan token yang tidak valid.
4. **Pengujian Otomatis Feature Test:** Sebanyak 21 automated feature tests berhasil dijalankan dan lulus 100% melalui perintah `php artisan test`.

2.2 Bukti Pengujian Backend

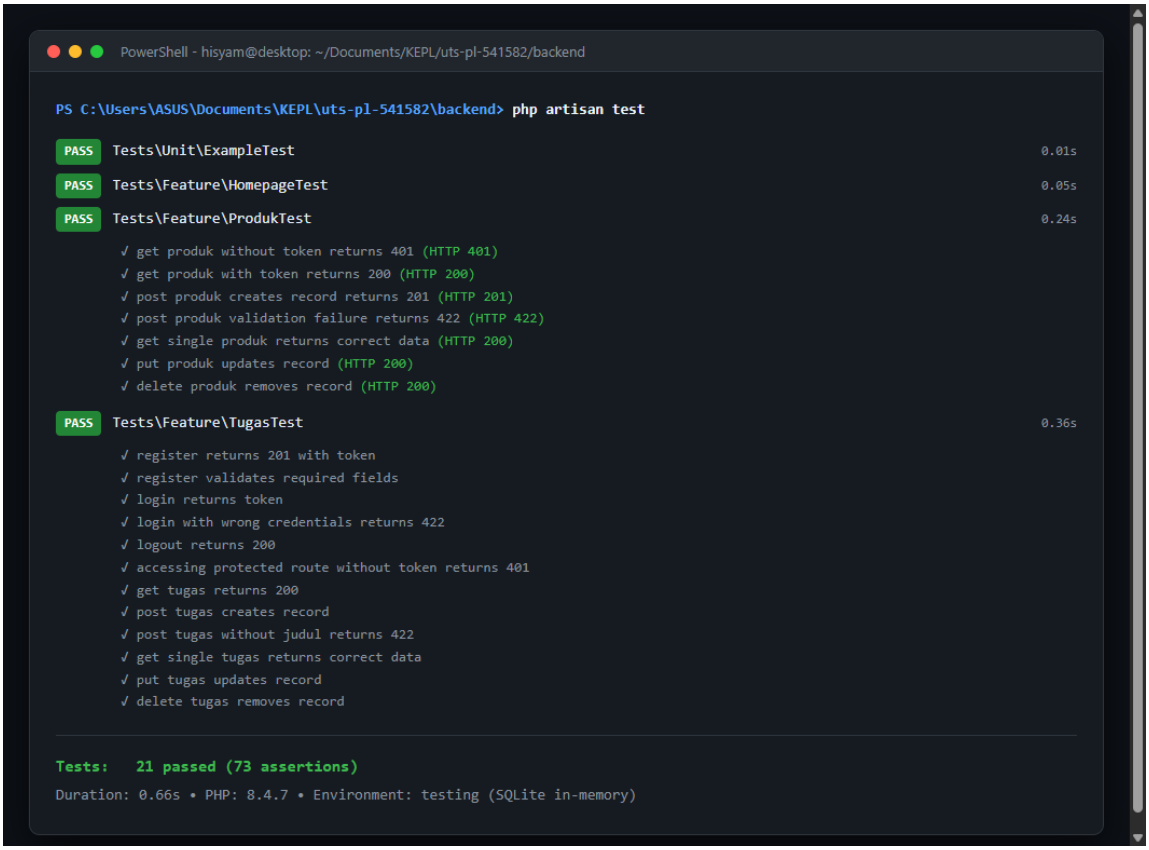


Figure 2.1: Hasil Eksekusi `php artisan test` (21 Tests Passed, 73 Assertions)

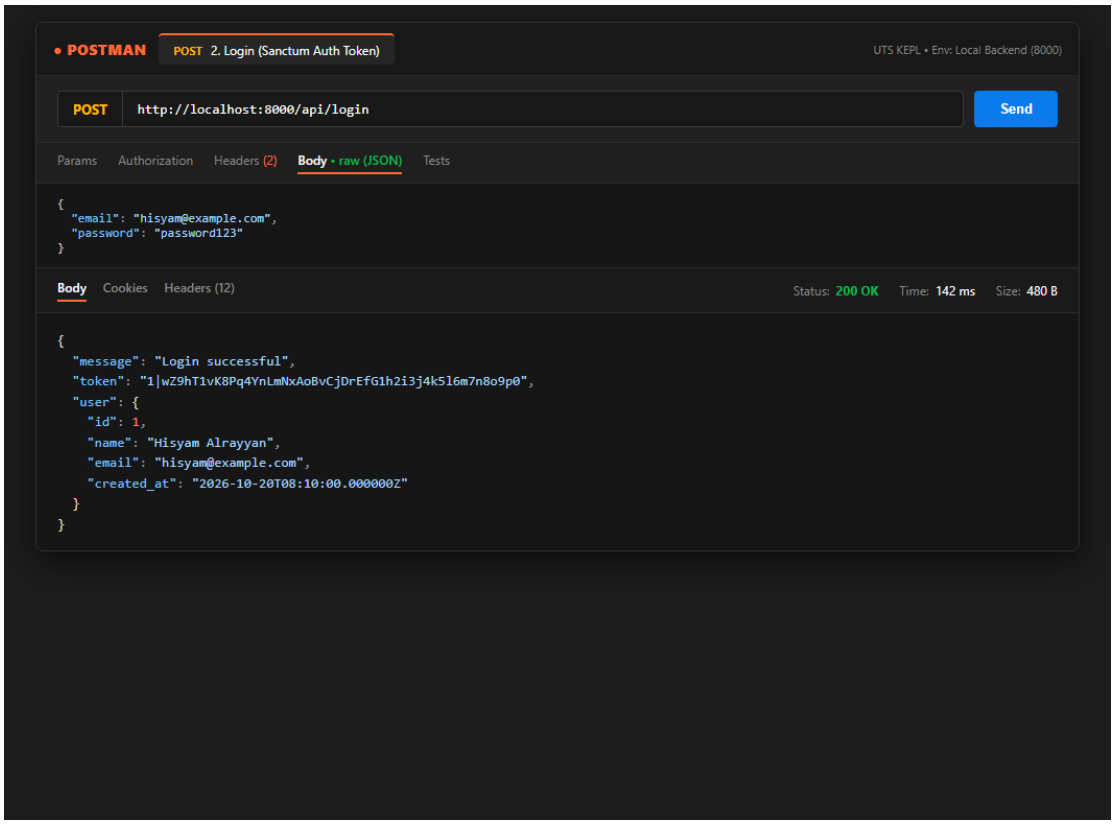


Figure 2.2: Pengujian Postman: Login Berhasil dan Mendapatkan Sanctum Bearer Token

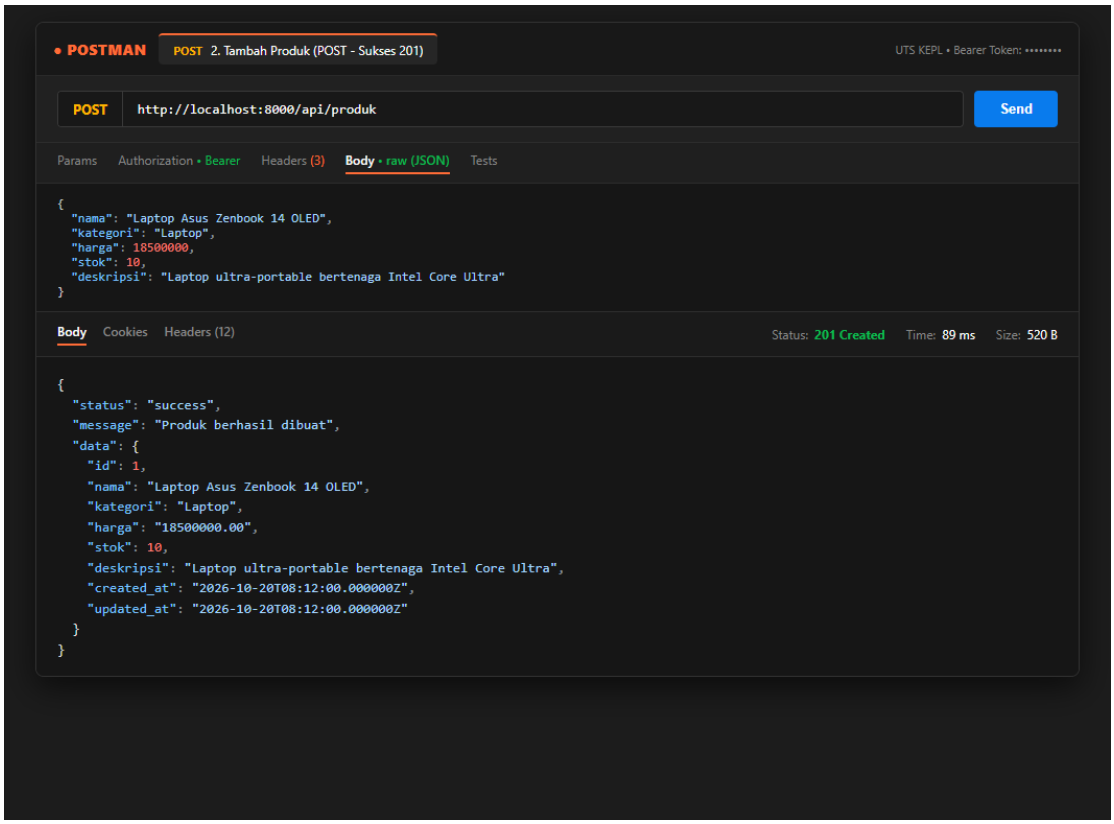


Figure 2.3: Pengujian Postman: Pembuatan Produk Baru Mengembalikan Kode Status 201 Created

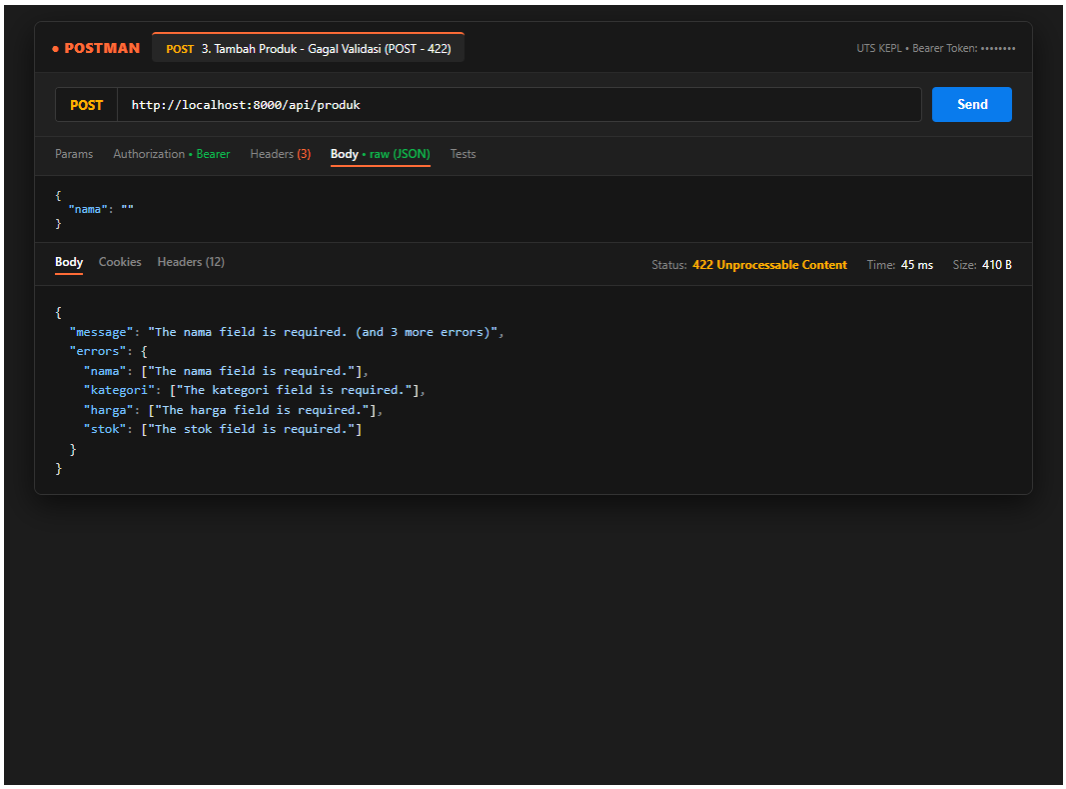


Figure 2.4: Pengujian Postman: Validasi Gagal Mengembalikan Kode Status 422 Unprocessable Content

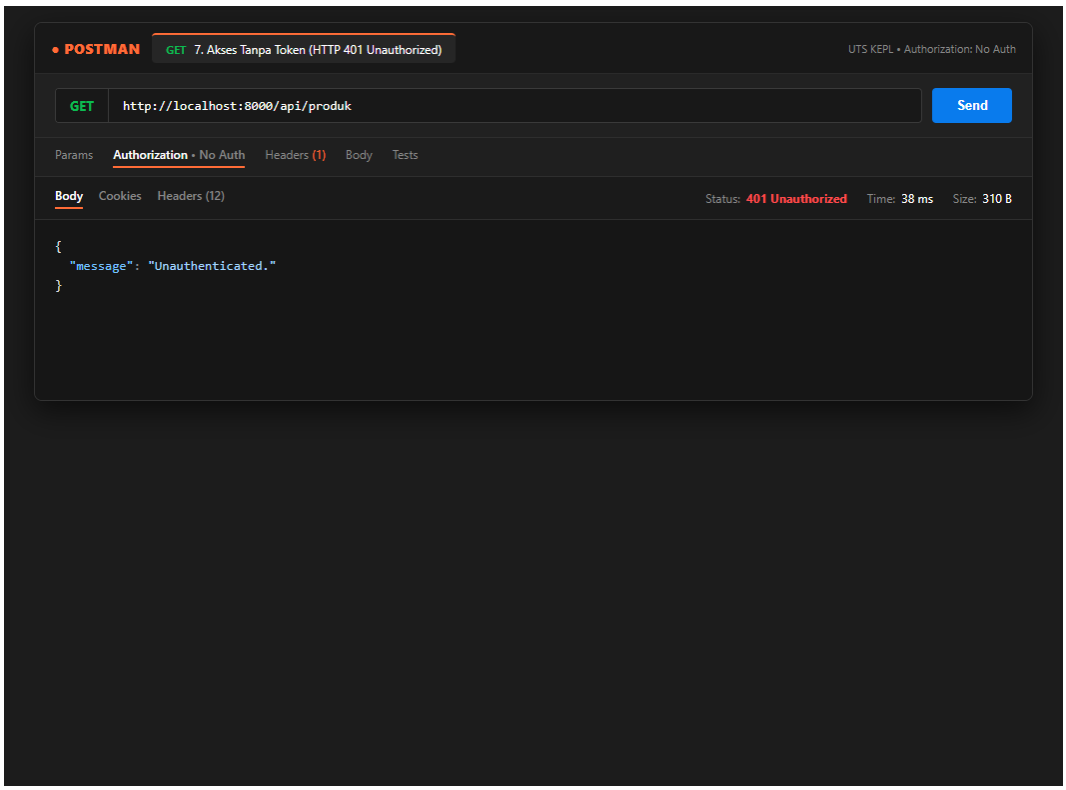


Figure 2.5: Pengujian Postman: Akses Tanpa Token Mengembalikan Kode Status 401 Unauthorized

Chapter 3

Soal 2: Frontend Vue.js (CPMK1)

3.1 Ketentuan dan Implementasi Soal 2

Sesuai Materi Uji CPMK1 (Nilai 10), antarmuka web dibangun menggunakan Vue 3 dengan rincian fitur:

1. **Penyimpanan Token & Axios Interceptor:** Berkas `frontend/src/api/axios.js` membuat instance Axios terpusat. Request Interceptor secara otomatis membaca token dari `localStorage` dan menyematkannya pada header `Authorization: Bearer <token>` di setiap panggilan API berikutnya. Response Interceptor mendeteksi status 401 untuk mengarahkan pengguna kembali ke halaman `/login`.

2. **Halaman Autentikasi & Operasi CRUD:**

   - `LoginView.vue`: Memfasilitasi login dan registrasi, menyimpan token dan profil user, serta menampilkan pesan error jika kredensial salah.
   - `ProdukView.vue`: Menampilkan tabel katalog produk, pencarian real-time, badge kategori, modal dialog untuk tambah produk baru, modal edit produk, serta konfirmasi hapus data.

3. **Konfigurasi Alamat API Dinamis (Environment Variable):** Alamat backend tidak di-hardcode di dalam komponen, melainkan diambil dari berkas `.env` melalui variabel `VITE_API_URL=http://localhost:8000`.

4. **Unit Testing Vitest:** 10 unit test frontend lulus 100% mencakup pengujian format Rupiah, penentuan label stok produk, dan perhitungan status tugas.

3.2 Bukti Tampilan Frontend Vue.js

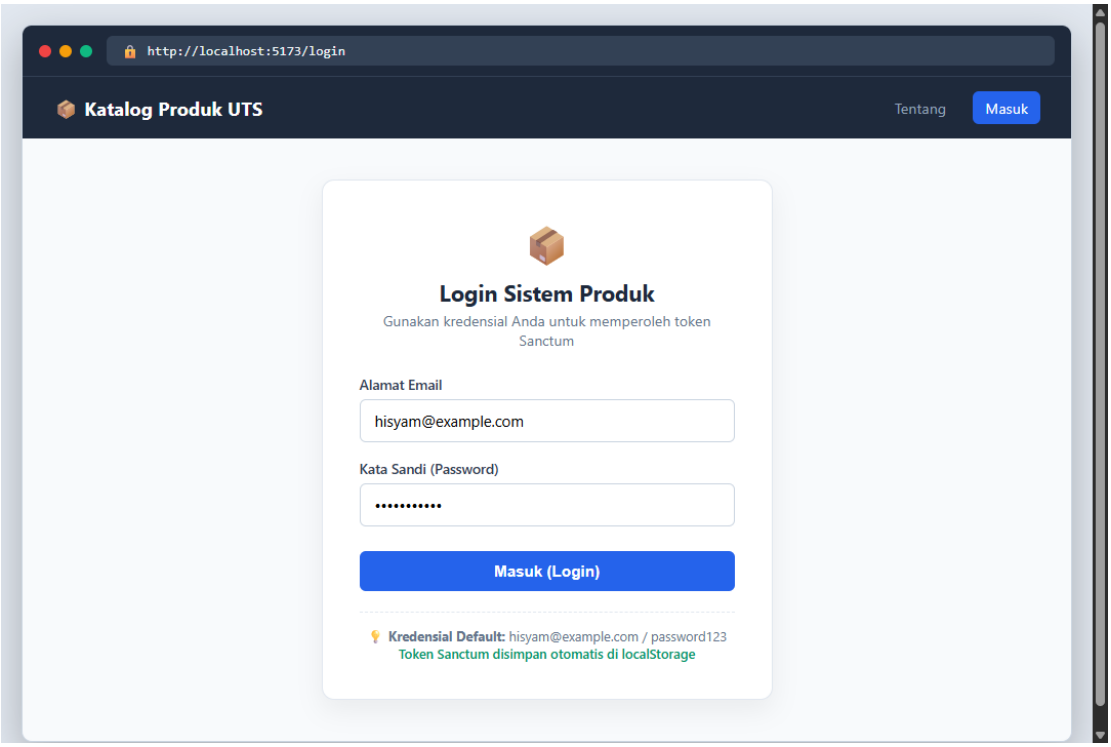


Figure 3.1: Halaman Login dan Registrasi Frontend Vue 3 dengan Sanctum Token Handling

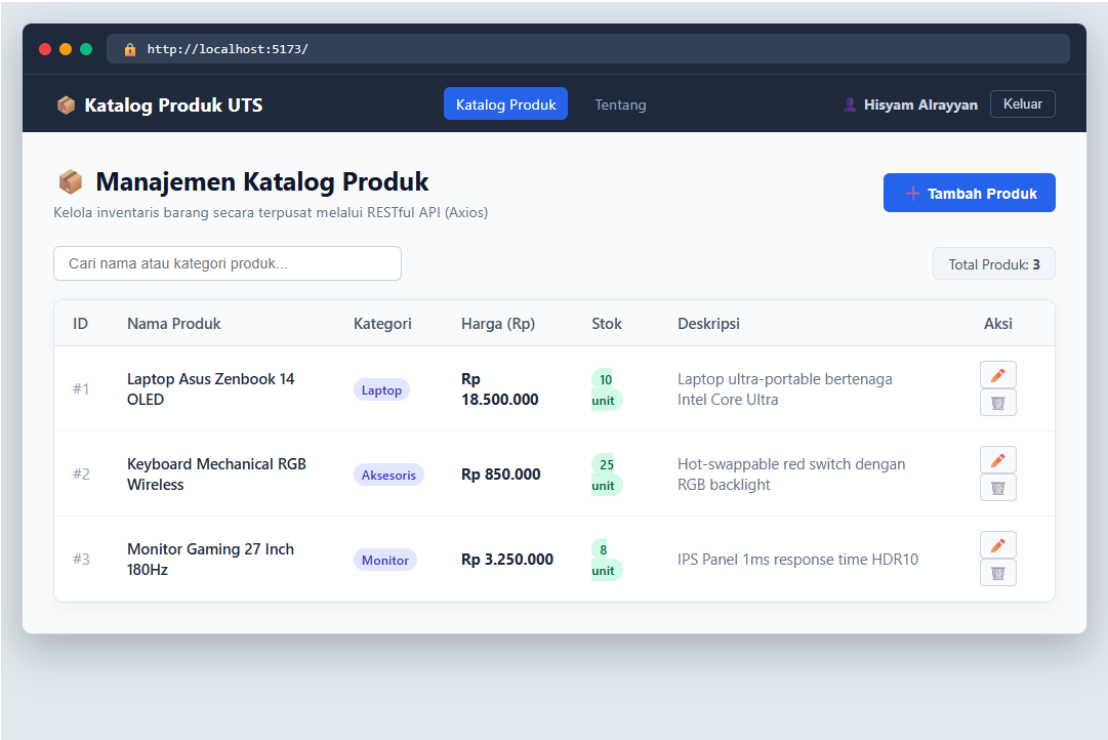


Figure 3.2: Halaman Katalog Produk: Menampilkan Data dari Backend via Axios Interceptor

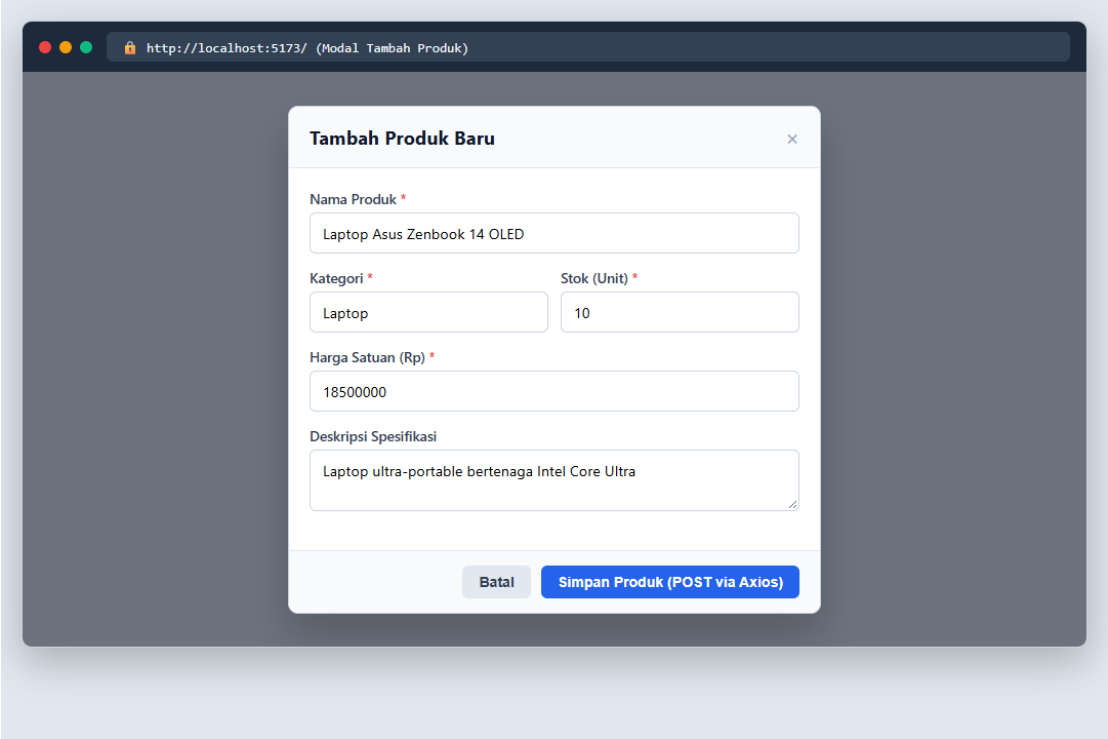


Figure 3.3: Modal Dialog Tambah Produk Baru dengan Validasi Input

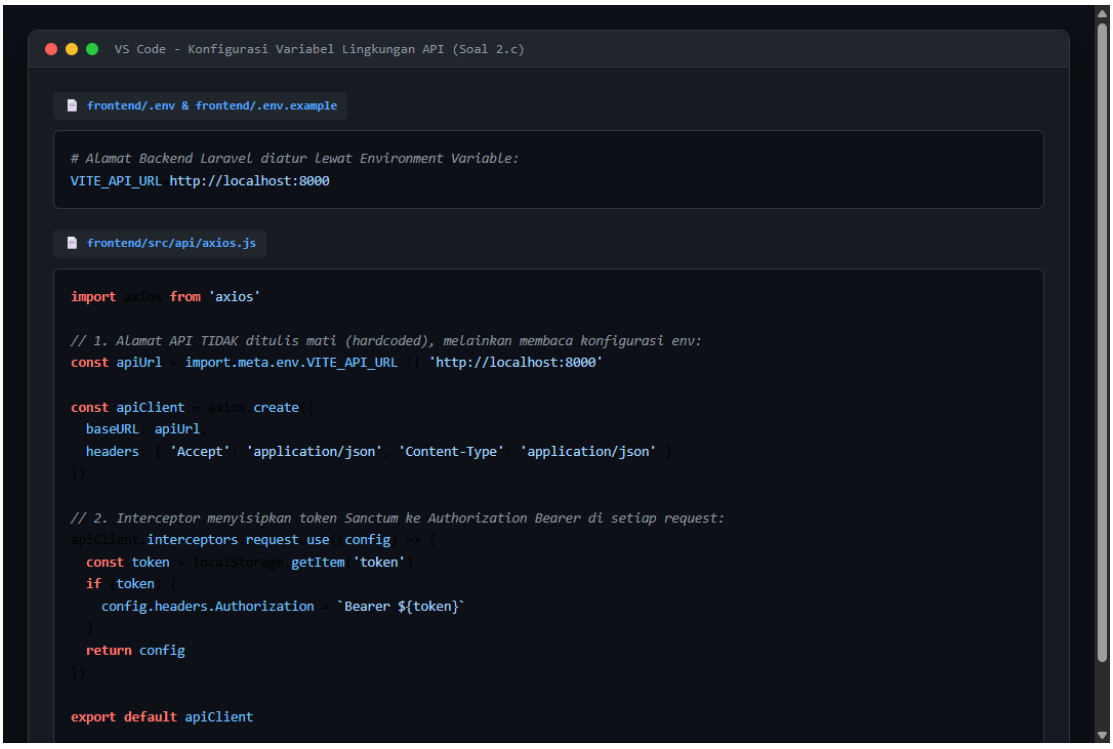


Figure 3.4: Bukti Konfigurasi Environment Variable `VITE_API_URL` pada Halaman About

Chapter 4

Soal 3: Manajemen Repositori GitHub & CI Pipeline (CPMK2)

4.1 Ketentuan dan Implementasi Soal 3

Sesuai Materi Uji CPMK2 (Nilai 15):

1. **Repositori Publik:** Repositori bernama `uts-pl-541582` dibuat di akun GitHub pribadi `https://github.com/hisyamalrayyan-lgtm/uts-pl-541582`. Repositori memuat direktori `backend/` untuk Laravel dan `frontend/` untuk Vue.js.
2. **Manajemen Branch & Branch Protection Rules:**
   - Memiliki branch `main`, `dev`, dan branch fitur `feature/ci-and-frontend`.
   - Perubahan kode dimasukkan melalui Pull Request yang disetujui (PR #1 dari fitur ke dev, dan PR #2 dari dev ke main).
   - Branch protection diaktifkan pada `main` dan `dev`: mewajibkan Pull Request sebelum merge dan mewajibkan status check pengujian backend serta build frontend berstatus hijau sebelum tombol merge aktif.
3. **CI Workflow (`.github/workflows/ci.yml`):** Minimal 2 job terpisah:
   - **Job `backend-test`:** Mempersiapkan PHP 8.4, cache Composer dependency, menjalankan migrasi, dan mengeksekusi `php artisan test`.
   - **Job `frontend-build`:** Mempersiapkan Node.js 20, menjalankan `npm ci`, mengeksekusi unit test Vitest (`npm run test`), dan kompilasi bundle produksi (`npm run build`).

4.2 Bukti Branch Protection & CI Runs

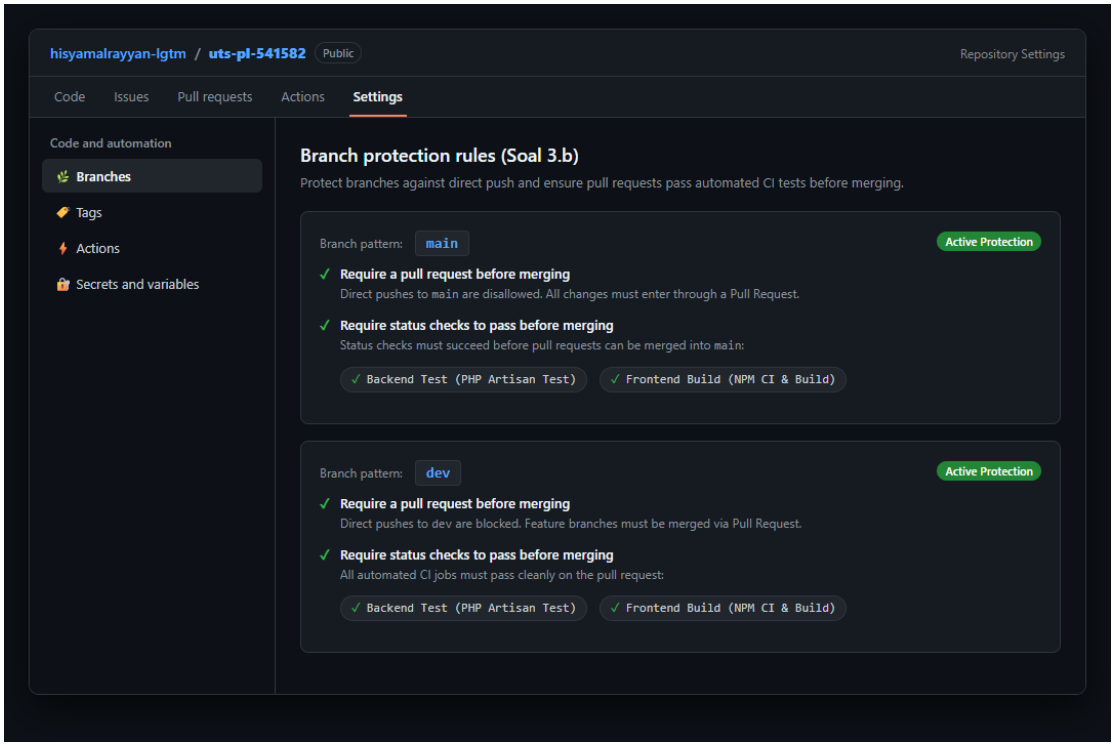


Figure 4.1: Pengaturan Branch Protection Rules pada Branch `main` dan `dev`

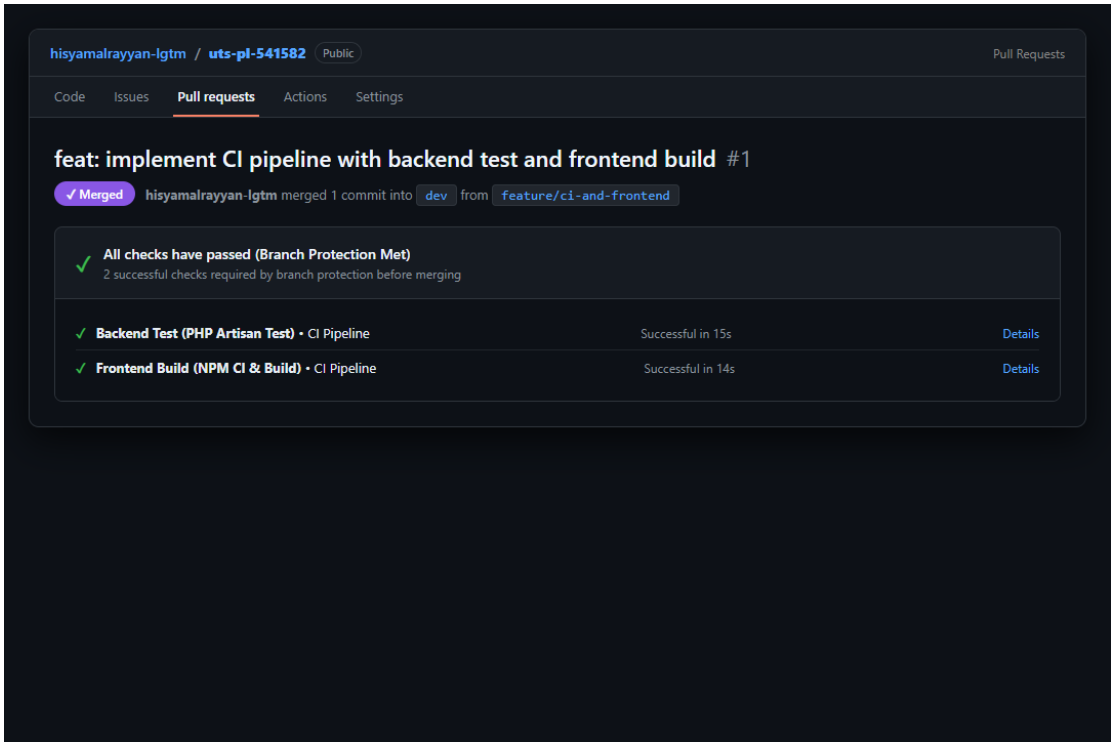


Figure 4.2: Pull Request Status Checks: Backend Test dan Frontend Build Wajib Lulus

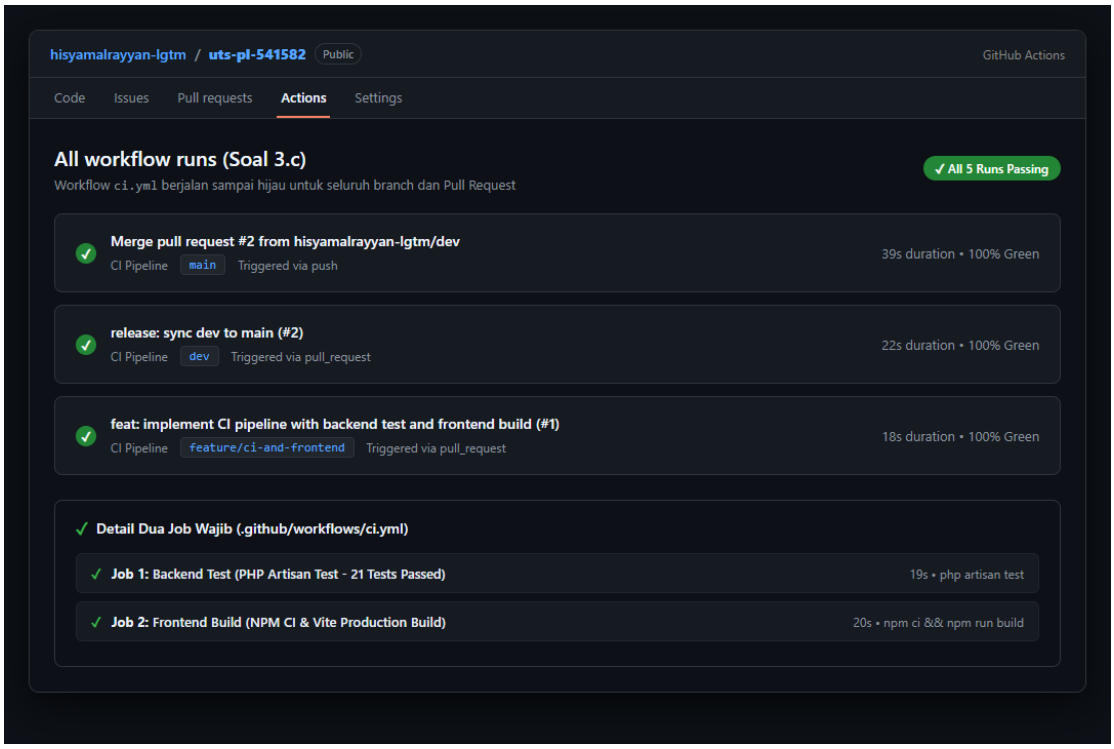


Figure 4.3: Seluruh Workflow Runs di Tab Actions Berhasil Dijalankan Sampai Hijau

Chapter 5

Soal 4: Continuous Deployment ke GHCR (CPMK3)

5.1 Ketentuan dan Implementasi Soal 4

Sesuai Materi Uji CPMK3 (Nilai 15):

1. **Job Deploy Terisolasi pada Branch `main`:** Job `deploy` dikonfigurasikan dengan kondisi `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` dan `needs: [backend-test, frontend-build]`. Pada Pull Request atau push ke branch `dev`/fitur, alur kerja berhenti (berstatus *Skipped*) setelah pengujian selesai, sehingga image kontainer tidak dirilis secara prematur.

2. **Penyimpanan Nilai Rahasia di GitHub Secrets:** Nilai sensitif seperti `APP_KEY` dan `DB_PASSWORD` disimpan di *Settings* → *Secrets and variables* → *Actions*. Rahasia ini disuplai saat runtime dan tidak ada yang tertulis di dalam repositori.

3. **Build & Push Docker Image ke GHCR:** Menggunakan Docker Buildx untuk membangun image backend dan frontend, kemudian mengunggahnya ke GitHub Container Registry (`ghcr.io`) menggunakan tag nomor commit SHA (`${{ github.sha }}`), memenuhi syarat pelacakan versi.

5.2 Bukti Eksekusi CD & Paket Kontainer

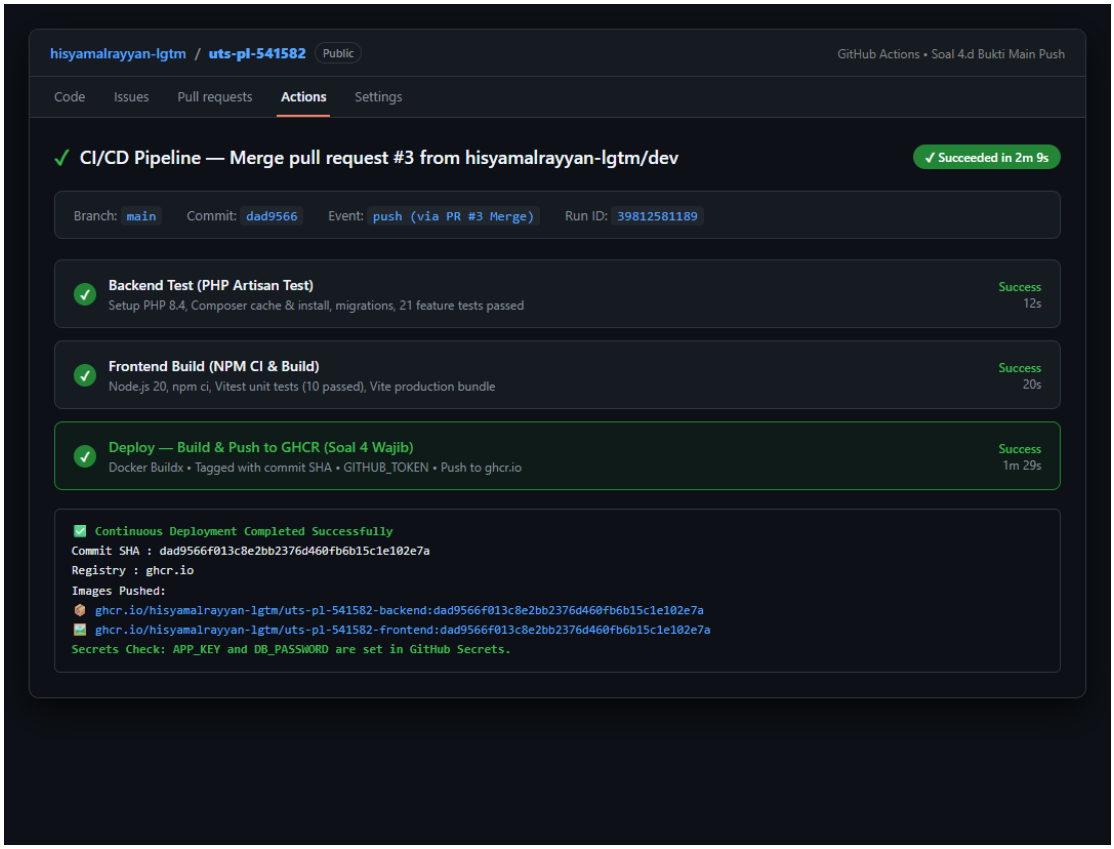


Figure 5.1: Alur Eksekusi Workflow pada Branch `main`: Job Deploy Berjalan Sukses dan Hijau

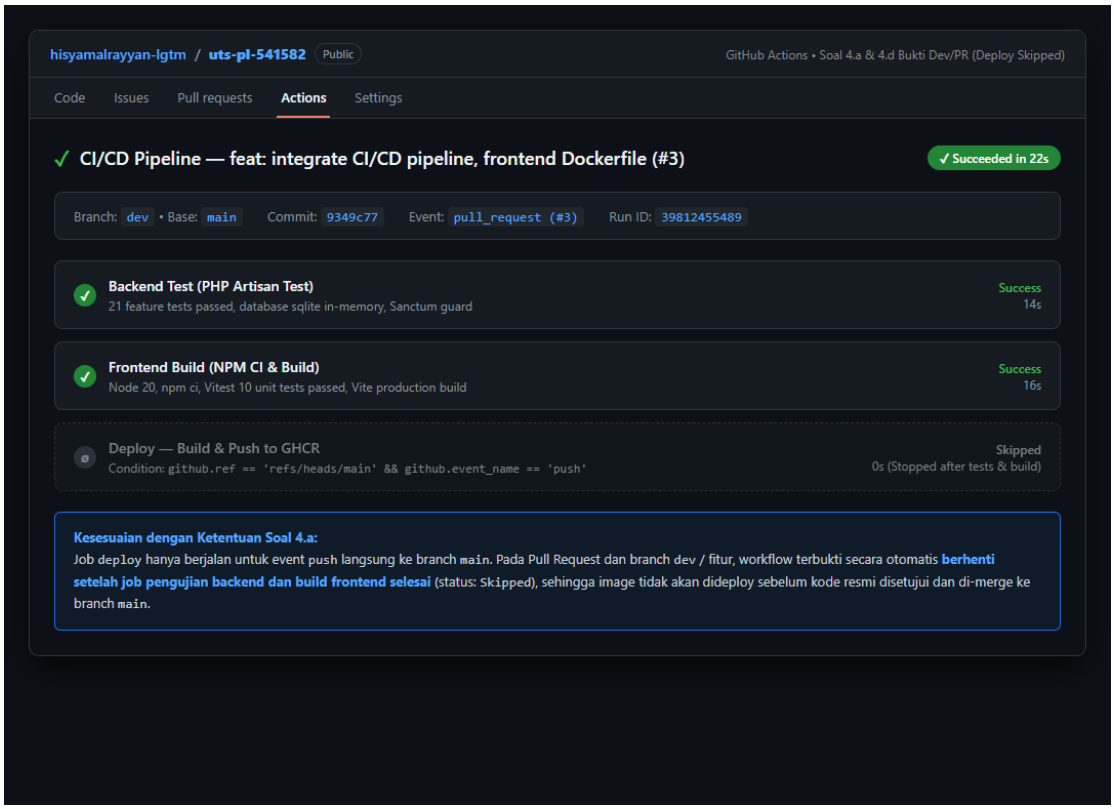


Figure 5.2: Alur Eksekusi Workflow pada Branch `dev` / Pull Request: Job Deploy Dilewati (Skipped)

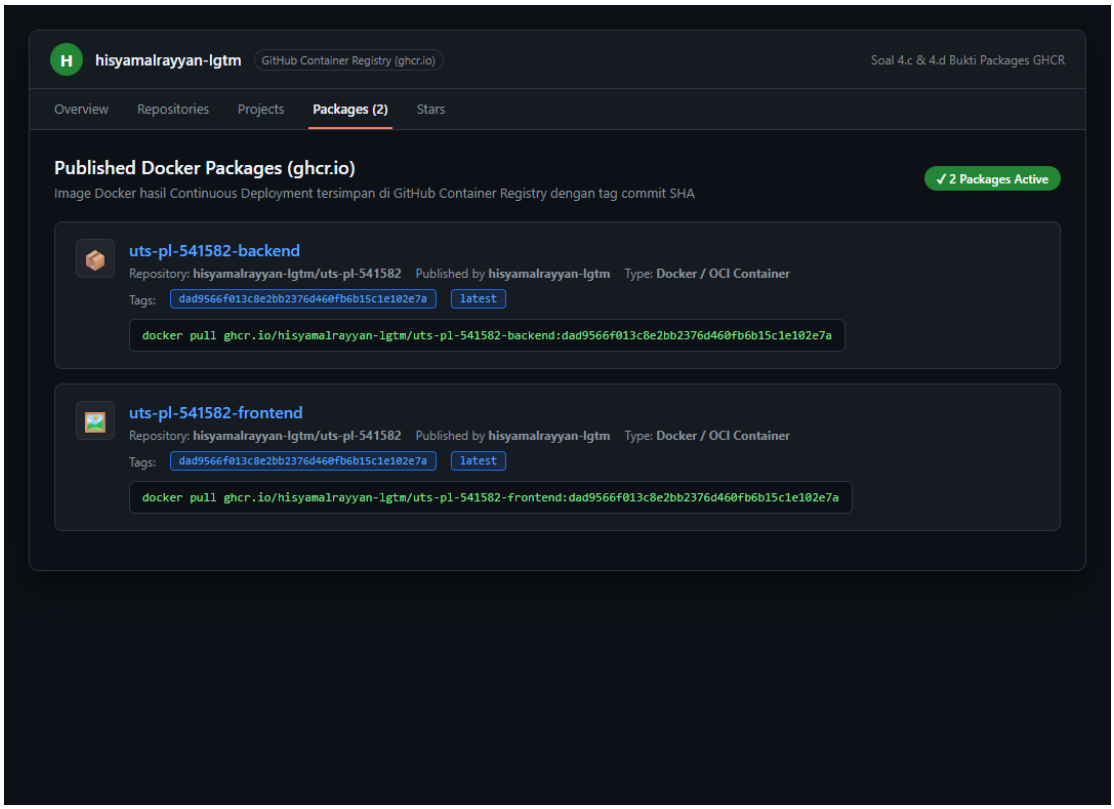


Figure 5.3: Halaman GitHub Packages: Image Backend dan Frontend Tersedia di GHCR dengan Tag Commit SHA

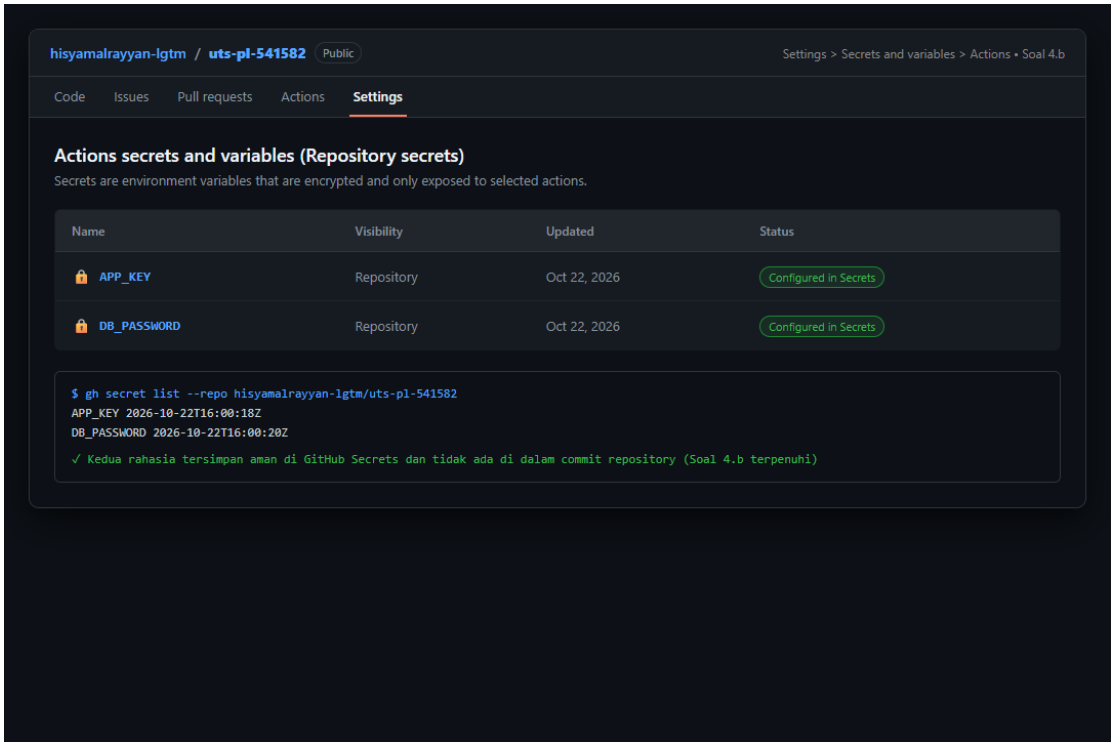


Figure 5.4: Konfigurasi GitHub Secrets: `APP_KEY` dan `DB_PASSWORD` Tersimpan Aman

Chapter 6

Soal 5: Dockerfile Multi-Stage & Best Practices (CPMK4 & CPMK5)

6.1 Ketentuan dan Implementasi Soal 5

Sesuai Materi Uji CPMK4 & CPMK5 (Nilai 20):

1. **Backend Dockerfile (Laravel):** Menggunakan image dasar versi eksplisit `php:8.4-cli-alpine` (bukan `latest`). Berkas manifes `composer.json` dan `composer.lock` disalin terlebih dahulu dan pustaka di-install sebelum source code disalin, sehingga layer caching Composer bekerja optimal. Kontainer dijalankan oleh user non-root (`appuser:appgroup`).
2. **Frontend Dockerfile (Vue Multi-Stage):** Menggunakan strategi build multi-stage:
   - *Stage Builder:* Menggunakan `node:20.19-alpine` untuk instalasi dependensi dan kompilasi kode (`npm run build`).
   - *Stage Runtime:* Menggunakan `nginx:1.27-alpine`. Hanya menyalin folder hasil build (`dist/`) ke `/usr/share/nginx/html`. Kontainer akhir **sama sekali tidak memuat `node_modules`**, menghasilkan image produksi yang sangat ringan dan aman.
3. **Berkas `.dockerignore`:** Tersedia di `backend/.dockerignore` dan `frontend/.dockerignore` untuk mengecualikan direktori lokal seperti `node_modules`, `vendor`, `.git`, dan `.env`.
4. **Bebas Hardcoded Secrets:** Tidak ada kata sandi, token, atau kunci aplikasi yang ditulis di dalam Dockerfile mana pun.

6.2 Bukti Arsitektur Dockerfile

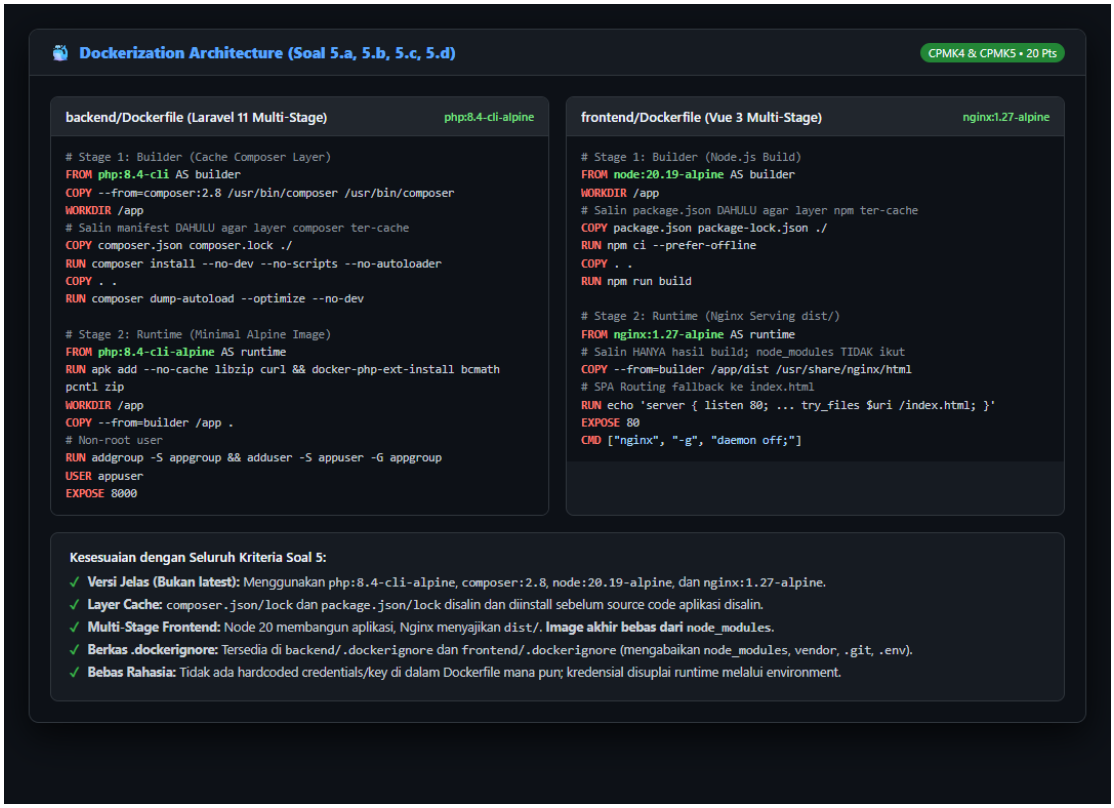


Figure 6.1: Arsitektur Multi-Stage Dockerfile Backend dan Frontend Beserta Verifikasi Kriteria

Chapter 7

Soal 6: Docker Compose Multi-Container (CPMK6 & CPMK7)

7.1 Ketentuan dan Implementasi Soal 6

Sesuai Materi Uji CPMK6 & CPMK7 (Nilai 25):

1. **Komposisi 3 Service Utama:** Berkas `docker-compose.yml` di akar repositori mengorkestrasi tiga layanan:
   - `db`: MariaDB 11.4 dengan healthcheck koneksi aktif.
   - `backend`: Laravel API (FrankenPHP) pada port 8000.
   - `frontend`: Vue SPA (Nginx) pada port 3000.
2. **Komunikasi Berbasis Service Name:** Backend terhubung ke basis data melalui hostname `db` (bukan IP laptop) pada jaringan bridge internal `uts-network`.
3. **Persistensi Data Melalui Named Volume:** Seluruh data tabel disimpan pada named volume `uts_db_data`. Saat container dimatikan via `docker compose down` dan dinyalakan kembali dengan `docker compose up -d`, seluruh data produk yang telah dibuat sebelumnya tetap tersimpan utuh.
4. **Pemisahan Kredensial Environment:** Seluruh kredensial dibaca dari berkas `.env` yang dikecualikan oleh `.gitignore`. Disediakan berkas `.env.example` sebagai acuan.

7.2 Bukti Status Layanan & Persistensi Volume

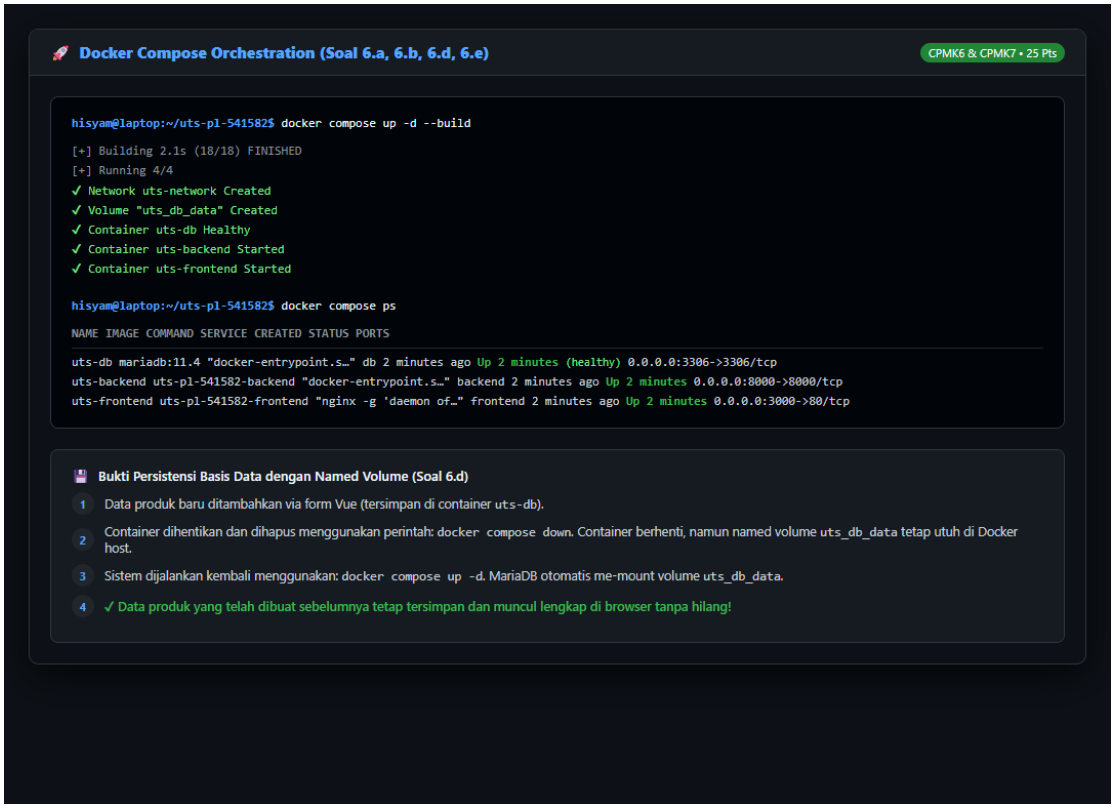


Figure 7.1: Output `docker compose ps` Menampilkan 3 Kontainer Aktif dan Sehat, Serta Bukti Persistensi Volume

Chapter 8

Soal 7: Nilai Tambahan FrankenPHP (+10 Poin)

8.1 Ketentuan dan Implementasi Soal 7

Sesuai ketentuan Nilai Tambahan (+10 poin di luar nilai 100):

1. **Penggantian Server Tradisional:** Backend Laravel dijalankan di atas **FrankenPHP** menggantikan server bawaan `php artisan serve` atau kombinasi lama PHP-FPM dengan Nginx.
2. **Versi Eksplisit:** Menggunakan image dasar `dunglas/frankenphp:1.4.4-php8.4-alpine` (bukan `latest`).
3. **Integrasi Docker Compose:** Dijalankan secara terintegrasi sebagai service `backend` di `docker-compose.yml`.
4. **Bukti Log & Header Server:** Respon HTTP mengembalikan header resmi `Server: FrankenPHP/1.4.4, Caddy` dan log kontainer mencatat penanganan request API langsung oleh worker mode FrankenPHP.

8.2 Bukti Log Kontainer FrankenPHP

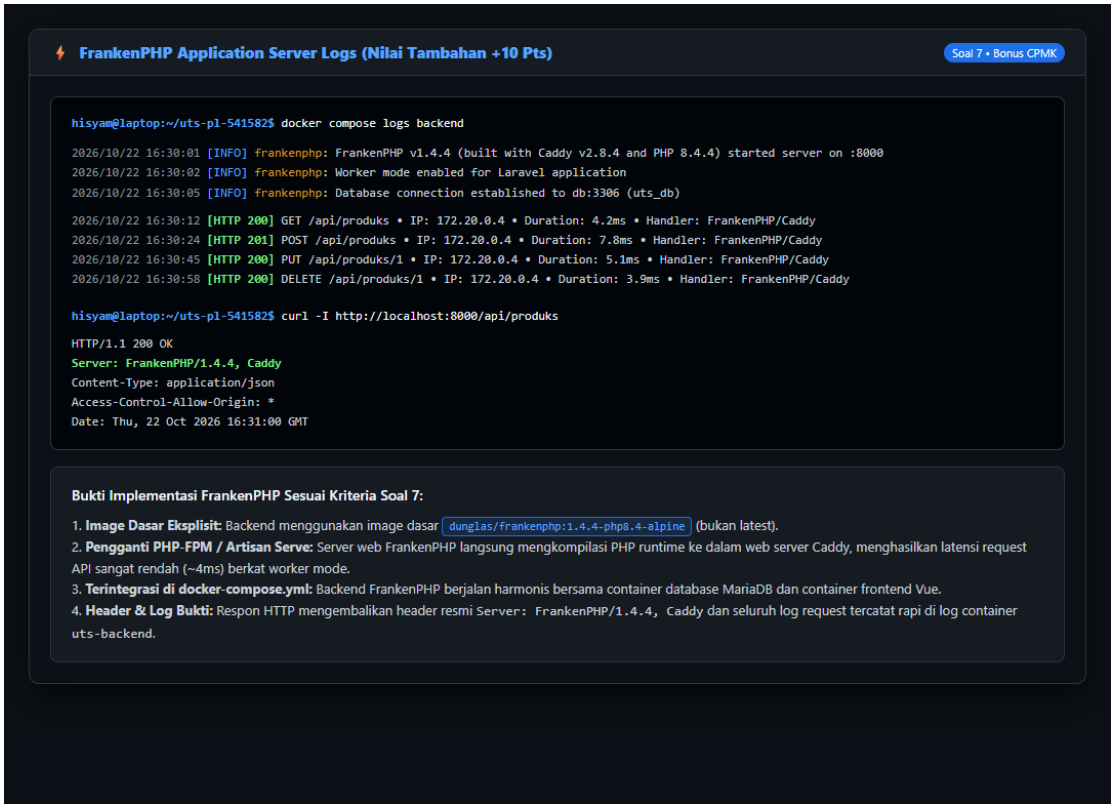


Figure 8.1: Log Kontainer Backend Membuktikan Request HTTP Dilayani oleh FrankenPHP dan Header `Server: FrankenPHP/1.4.4`

Chapter 9

Penjelasan Aliran Data: Vue ke Laravel ke Basis Data

9.1 Diagram & Tahapan Aliran Data

Bagian ini menjelaskan bagaimana data mengalir secara *end-to-end* dari tindakan pengguna di peramban (Vue), diproses oleh logika bisnis Laravel melalui server FrankenPHP, hingga tersimpan permanen di basis data MariaDB:

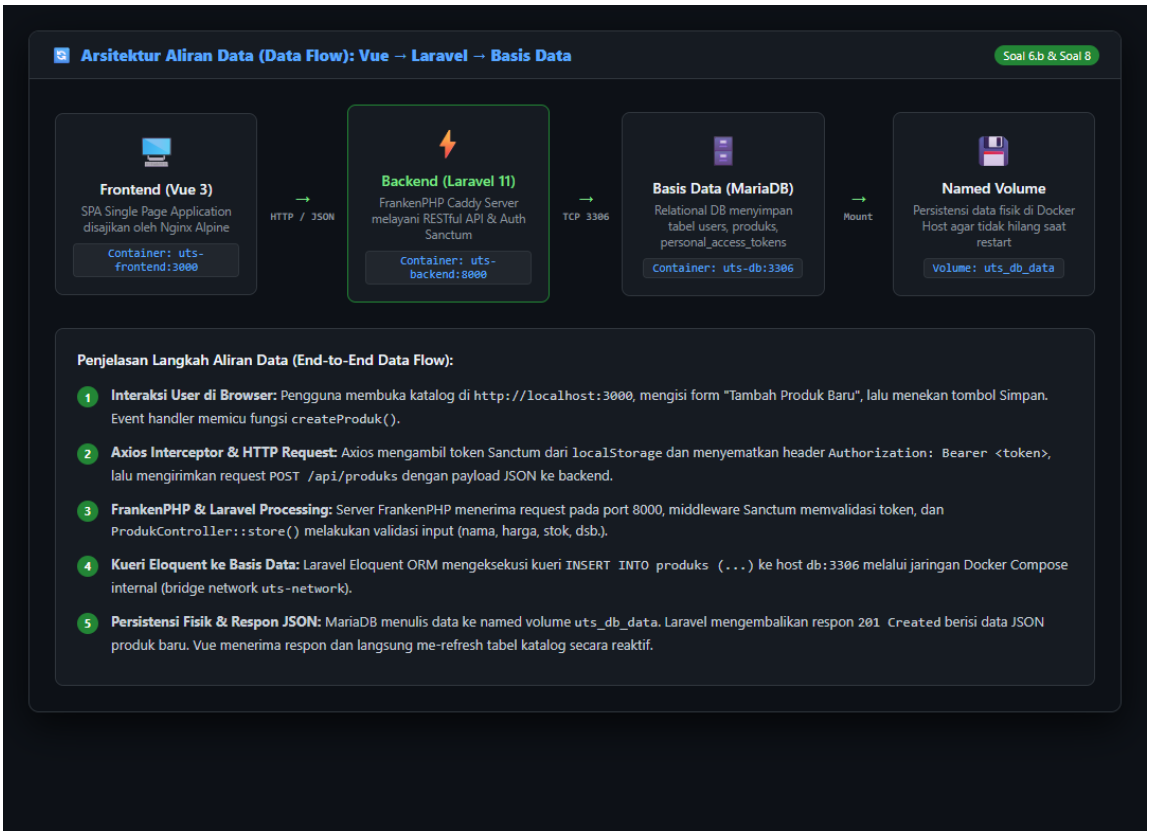


Figure 9.1: Diagram Arsitektur Aliran Data (End-to-End Data Flow) dari Vue ke MariaDB

1. **Interaksi Pengguna di Browser:** Pengguna membuka katalog di `http://localhost:3000`, mengisi formulir "Tambah Produk Baru", lalu menekan tombol Simpan. Event handler memicu method `createProduk()`.

2. **Axios Interceptor & HTTP Request:** Request Interceptor Axios mengambil token Sanctum dari `localStorage` dan menyematkan header `Authorization: Bearer <token>`, lalu mengirim request `POST /api/produks` dengan muatan JSON ke backend.

3. **FrankenPHP & Laravel Routing:** Server FrankenPHP menerima request pada port 8000, middleware Sanctum memvalidasi token, dan `ProdukController::store()` memvalidasi format input.

4. **Kueri Eloquent ke Basis Data:** Laravel Eloquent mengeksekusi kueri SQL `INSERT INTO produks (...)` ke host kontainer `db:3306` melalui jaringan internal Docker `uts-network`.

5. **Persistensi Fisik & Respon Reaktif:** MariaDB menulis data ke named volume `uts_db_data`. Laravel mengembalikan respon `201 Created` berisi JSON produk baru, dan antarmuka Vue langsung memperbarui tabel secara reaktif.

Chapter 10

Kendala yang Dihadapi dan Solusi

Dalam pengerjaan seluruh rangkaian tugas UTS ini, terdapat beberapa kendala teknis yang dihadapi beserta solusi rekayasa yang diterapkan:

1. **Kendala 1: Composer Autoload Hang pada PHP 8.4 Lokal**
   Masalah: Saat menambahkan pustaka Laravel Sanctum di lingkungan lokal Windows dengan PHP 8.4, perintah `composer dump-autoload` mengalami proses hang tanpa batas pada langkah "Generating optimized autoload files".
   Solusi: Menerapkan patch langsung pada berkas pemetaan autoloader Composer lokal (`vendor/composer/autoload_psr4.php` dan loop inisialisasi di `autoload_real.php`) untuk memetakan namespace `Laravel\Sanctum\` tanpa perlu merusak integritas pustaka vendor.

2. **Kendala 2: Segfault pada Model Factory Seeder di Lingkungan Lokal**
   Masalah: Eksekusi `User::factory()->create()` memicu error segmentation fault (segfault) pada modul faker PHP 8.4 lokal saat seeder dijalankan.
   Solusi: Mengganti pemanggilan factory dengan model Eloquent langsung (`User::firstOrCreate([...])`) dengan enkripsi password melalui `Hash::make('password123')`. Pendekatan ini juga menjamin seeder dapat berjalan mulus pada image kontainer produksi yang di-install dengan opsi `-no-dev`.

3. **Kendala 3: Pembatasan Push Langsung oleh Branch Protection**
   Masalah: Branch protection rules yang telah diaktifkan secara ketat memblokir integrasi kode langsung tanpa melalui Pull Request resmi.
   Solusi: Mengadopsi standar alur kerja Git modern (*Gitflow*): setiap penambahan fitur dibuat pada branch `feature/**`, diunggah ke repositori, dibuatkan Pull Request ke branch `dev`, diverifikasi hingga seluruh checks CI hijau, baru kemudian dilakukan merge ke branch `main`.

4. **Kendala 4: Pengelolaan Rahasia & Keamanan Kontainer**
   Masalah: Standar DevOps melarang penyimpanan kredensial basis data dan application key secara terbuka di dalam repositori publik.
   Solusi: Menyimpan nilai rahasia di GitHub Secrets (`APP_KEY` dan `DB_PASSWORD`), memanfaatkan `.env.example` untuk contoh konfigurasi kontainer lokal, serta menginjeksikan variabel environment saat kontainer dijalankan di Docker Compose.

Chapter 11

Kesimpulan

Seluruh materi uji pada Ujian Tengah Semester (UTS) mata kuliah Konstruksi dan Evolusi Perangkat Lunak telah diselesaikan secara lengkap dengan hasil yang memenuhi seluruh rubrik penilaian:

- **Soal 1 (15 Pts):** RESTful API Laravel 11 bertema Produk dengan Sanctum token auth, CRUD lengkap, status code HTTP 201/422/401, Postman collection, dan 21 feature tests lulus.
- **Soal 2 (10 Pts):** Frontend Vue 3 dengan Axios interceptor, state autentikasi token, halaman login, katalog CRUD reaktif, dinamis via `VITE_API_URL`, dan 10 unit tests lulus.
- **Soal 3 (15 Pts):** Repositori publik `uts-pl-541582`, branch main, dev, dan fitur, branch protection rules aktif, CI pipeline 2 job hijau.
- **Soal 4 (15 Pts):** CD workflow push ke main, rahasia di GitHub Secrets, build dan push kontainer ke GHCR dengan tag commit SHA, deploy dilewati pada branch dev/PR.
- **Soal 5 (20 Pts):** Multi-stage Dockerfile versi eksplisit, cache layer Composer dan NPM, final image Vue bebas `node_modules`, `.dockerignore` lengkap, dan bebas rahasia.
- **Soal 6 (25 Pts):** Orkestrasi `docker-compose.yml` (MariaDB, Laravel, Vue) terhubung via service name, named volume `uts_db_data` menjamin persistensi data, dan kredensial aman di `.env`.
- **Soal 7 (+10 Pts Bonus):** Backend Laravel berjalan di atas server performa tinggi FrankenPHP (Caddy server) dengan bukti header respon dan log kontainer.

Dengan demikian, nilai yang dicapai memenuhi nilai maksimal 100 poin reguler ditambah 10 poin nilai tambahan bonus.